



Describe your data

```
Type Project {
  query: String!
  tagline: String!
  contributors: [User]!
}
```

Ask for what you want

```
{
  project(name: "GraphQL") {
    tagline
  }
}
```

Get predictable results

```
{
  "project": {
    "tagline": "A query language for APIs"
  }
}
```

Query

```
{
  hero {
    name
    height
  }
}
```

Response

```
{
  "hero": {
    "name": "Luke Skywalker",
    "height": 1.72
  }
}
```

🔗 What is GraphQL ?
GraphQL (Graph Query Language) is an open-source data query and manipulation language for APIs. Initially developed by Facebook to optimize data fetching. Similar to REST API but provides flexibility in choosing what data to fetch.

Uses:
Query → Fetch data.
Mutation → Modify (Create, Update, Delete) data.

🔍 What is a GraphQL Query?
Used to retrieve data from the server.
Allows clients to request only the required fields, avoiding over-fetching and under-fetching.

🔧 What is a GraphQL Mutation?
Used to modify data (Insert, Update, Delete).
Provides a structured way to perform CRUD operations in GraphQL.

📄 What is a GraphQL Schema?
Defines how data is structured and queried.
Provides flexibility to API consumers in selecting response attributes.
Defined using .graphql files.



Key Features:

Acts as a contract between API consumers and providers.
Specifies data types and attributes for queries and mutations.
Contains root types (Query and Mutation) as entry points.

Feature	GraphQL	REST API
Data Fetching	Fetch only required fields	Fixed response with all attributes
Over-fetching & Under-fetching	❌ Eliminated	❌ Common problem
Endpoints	Single endpoint (/graphql)	Multiple endpoints for each resource
Schema Requirement	Requires a schema file	No schema needed
HTTP Methods	Uses Query & Mutation	Uses GET, POST, PUT, DELETE

Category	Concept	Explanation	Example
🔍 Introduction	What is GraphQL ?	GraphQL is a query language for APIs that allows fetching only required data. Developed by Facebook.	Example: Instead of calling multiple REST APIs, GraphQL fetches all required data in one request.
	Why Use GraphQL ?	- No over-fetching or under-fetching - Uses a single API endpoint - Faster and flexible data retrieval.	Instead of calling /users and /users/orders separately in REST, GraphQL fetches everything in one request.
	GraphQL vs REST API	GraphQL: Fetch only needed data with a single query.REST: Fixed response format, multiple endpoints needed.	GraphQL Query: { user{id: "1"} { name, orders { id } } } REST Calls: /users/1 + /users/1/orders
📄 GraphQL Schema	What is a Schema?	Defines what data is available and how it can be queried.	Written in .graphqls file.
	Schema Structure	- Query: Fetch data. - Mutation: Modify data. - Types: Define objects.	graphql type User { id: ID! name: String! email: String! }
	Root Types	- Query: For fetching data. - Mutation: For modifying data. - Subscription: For real-time updates.	graphql type Query { user(id: ID!) : User }
🔧 Setup in Spring Boot	Required Dependencies	To use GraphQL in Spring Boot, add:	<pre><xml><dependency> <groupId>org.springframework.boot</groupId> <artifactId>spring-boot-starter-graphql</artifactId> </dependency></pre>
	GraphQL Endpoint	Default endpoint is /graphql. Can be changed in application.properties.	spring.graphql.path=/graphql-api
🔍 Queries (Fetching Data)	What is a Query?	Queries are used to fetch data from the server.	graphql query { user(id: "1") { name email } }
	Java Query Resolver	Uses GraphQLQueryResolver to fetch data in Java.	java @QueryMapping public User getUser(@Argument String id) { return userService.findById(id); }
🔧 Mutations (Modifying Data)	What is a Mutation?	Used to modify data (Create, Update, Delete).	graphql mutation { createUser(name: "Aman", email: "aman@example.com") { id name } }
	Java Mutation Resolver	Uses GraphQLMutationResolver to handle data modifications.	java @MutationMapping public User createUser(@Argument String name, @Argument String email) { return userService.create(name, email); }
📄 GraphQL Schema & Types	GraphQL Types	- Scalars: String, Int, Float, Boolean, ID, Objects & Lists	graphql type Product { name: String price: Float }
	Non-Nullable Fields	! means field cannot be null.	name: String! (Must have a value)
	Input Types	Used for structured mutation inputs.	graphql input UserInput { name: String! email: String! }
🔗 Advanced Features	Nested Queries	Fetch multiple related objects in a single request.	graphql query { user(id: "1") { name posts { title } } }
	Field Resolvers	Used to fetch nested fields dynamically.	java @SchemaMapping public List<Post> posts(User user) { return postService.findByUserId(userId); }
	Batched Queries	Execute multiple queries in a single request.	graphql query { user1: user(id: "1") { name } user2: user(id: "2") { name } }
	Subscriptions (Real-time Updates)	Enables live data updates using WebSockets.	graphql subscription { newPost { title content } }
🔒 Security	Authentication	Supports JWT, OAuth, API Key authentication.	Header: Authorization: Bearer <token>
	Authorization	Restrict access based on user roles.	java @PreAuthorize("hasRole('ADMIN')") @MutationMapping public void deleteUser(@Argument String id) { ... }
🧪 Testing GraphQL	GraphQL (Jl Tool)	In-browser tool for testing GraphQL APIs.	Enable in Spring Boot: spring.graphql.graphql.enabled=true
	Postman	Supports GraphQL testing under the GraphQL tab.	API Endpoint: http://localhost:8080/graphql
	Apollo Client	Popular GraphQL client for frontend apps.	Used in React, Angular, Vue.js.